FPT SCHOOL OF BUSINESS & TECHNOLOGY

AI - MACHINE LEARNING

PhD. Truc Thi Kim, Nguyen

093 293 9981 info@fsb.edu.vn https://fsb.edu.vn

Practical Aspects of Deep Learning

FSB

Applied ML is a highly iterative process

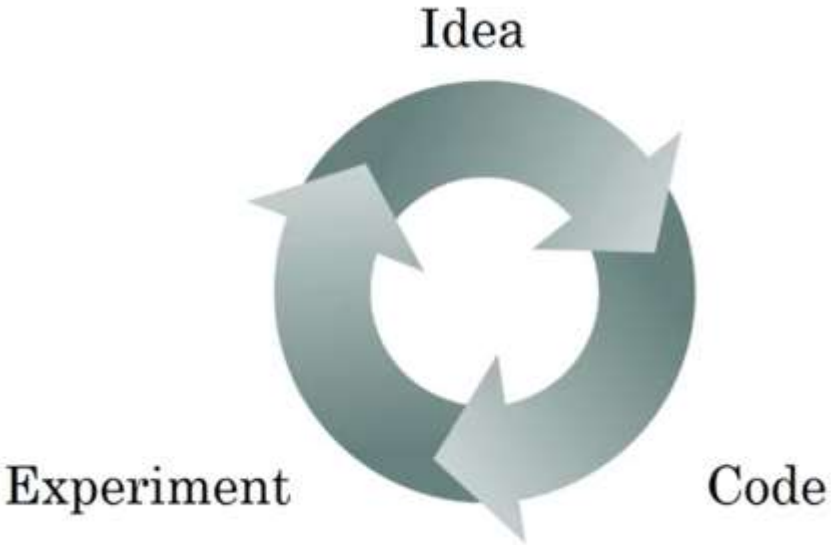
layers

hidden units

learning rates

activation functions

...



FSB

Train/ Dev/Test sets

- It's impossible to get all your hyperparameters right on a new application from the first time.
- So the idea is you go through the loop: Idea ==> Code ==> Experiment.
- You have to go through the loop many times to figure out your hyperparameters.
- Your data will be split into three parts:
  - Training set. (Has to be the largest set)
  - Hold-out cross validation set / Development or "dev" set.
  - Testing set.

FSB

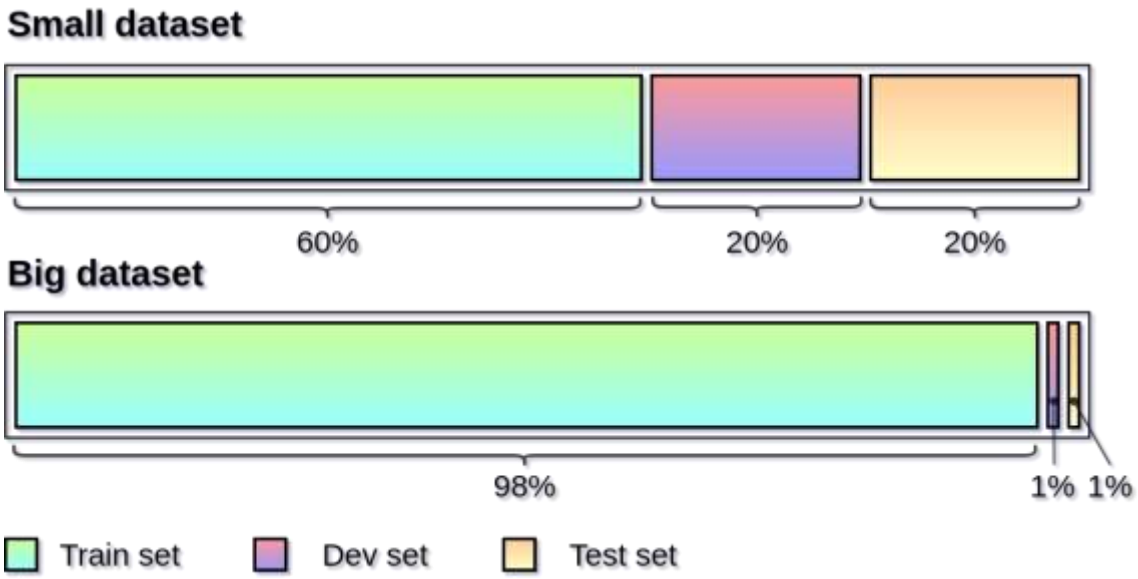
Train/ Dev/Test sets

- A training data set is a data set of examples used during the learning process and is used to fit the parameters (e.g., weights) of, for example, a classifier.
- A validation data set is a data set of examples used to tune the hyperparameters (i.e. the architecture) of a model. It is sometimes also called the development set or the "dev set".
- A test data set is a data set that is independent of the training data set, but that follows the same probability distribution as the training data set. If a model fit to the training data set also fits the test data set well, minimal overfitting has taken place (see figure below). A better fitting of the training data set as opposed to the test data set usually points to over-fitting.

FSB

How to split the sets based on size of dataset

- If size of the dataset is 100 to 1000000 ==> 60/20/20
- If size of the dataset is 1000000 to INF ==> 98/1/1 or 99.5/0.25/0.25



FSB

Mismatched train/test distribution

- Example:

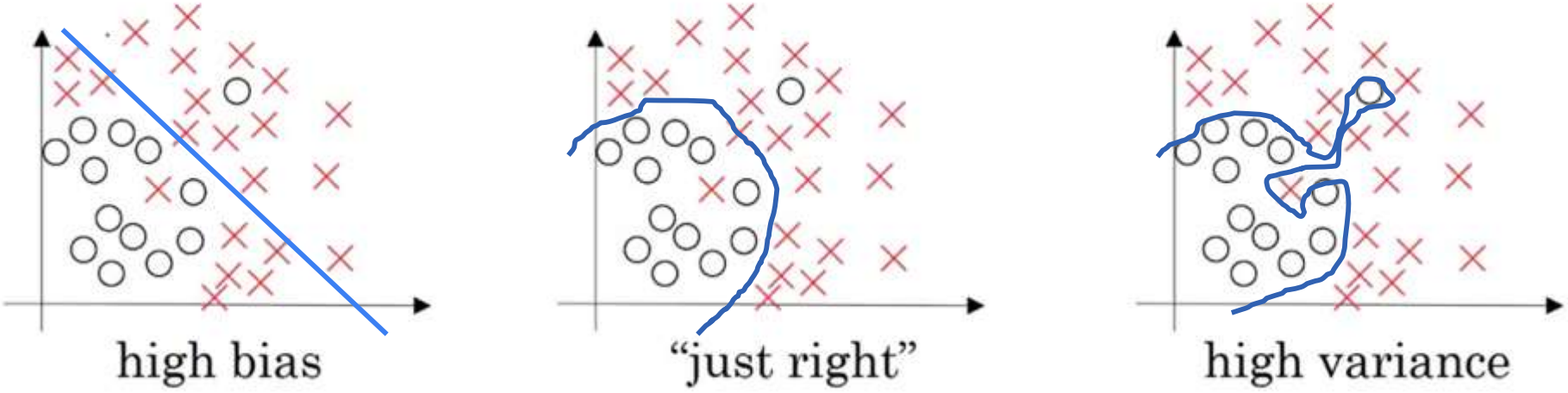
Training set:
Cat pictures from webpages

Dev/test sets:
Cat pictures from users using your app

Not having a test set might be okay. (Only dev set.)

FSB

Bias vs Variance



Underfitting

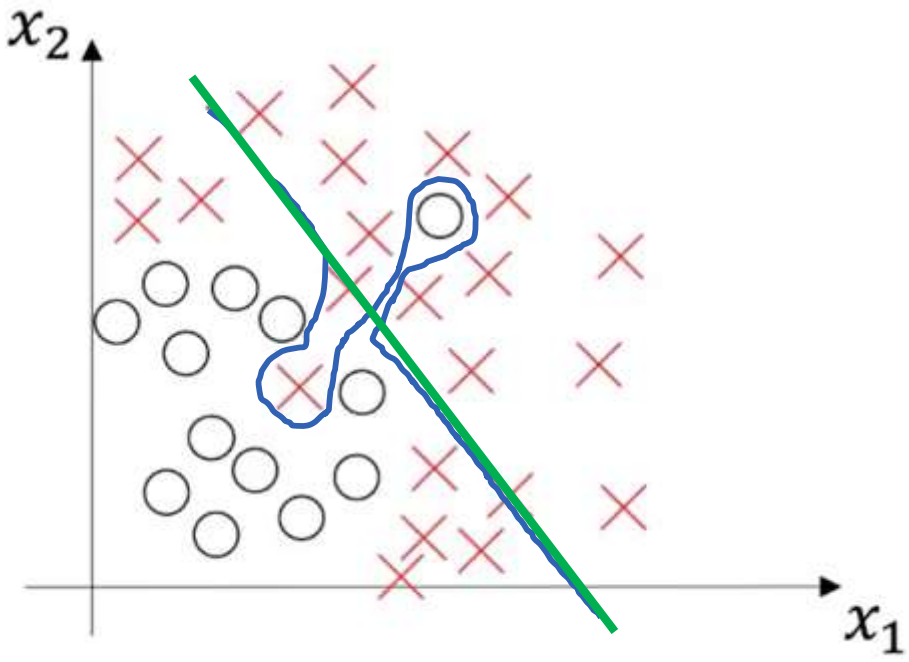
Overfitting

Bias vs Variance

Cat classification

Train set error	1 %	15%	15%	0.5%
Dev set error	11 %	16%	30%	1%
	High Variance	High Bias	High Bias High Variance	Low Bias Low Variance

High bias and high variance



Hint to know high bias or high variance or so on is that we should look at the training set error and dev set error.

FSB

Basic recipe for ML

High bias ?
(Training data performance)

Y →

- Bigger network
- Train longer

(NN architecture search)

N ↓

High variance ?
(Dev set performance)

Y →

- More data
- Regularization

(CNN architecture search)

N ↓

Done

BIAS – VARIANCE TRADEOFF

Methods to deal with BIAS – VARIANCE TRADEOFF

Regularization
Normalization
Weight initialization

FSB

Regularization methods

- Adding regularization to NN will help it reduce variance (overfitting)

- L2 norm – weight decay
- L1 norm
- Dropout
- Early stopping

FSB

L2 regularization for LR

- Regularization for logistic regression:

- The normal cost function that we want to minimize is:

$$J(w,b) = (1/m) * Sum(L(y(i),y'(i)))$$

- The L2 regularization version:

$$J(w,b) = (1/m) * Sum(L(y(i),y'(i))) + (lambda/2m) * Sum(|w[i]|^2)$$

- The L1 regularization version:

$$J(w,b) = (1/m) * Sum(L(y(i),y'(i))) + (lambda/2m) * Sum(|w[i]|)$$

- The L1 regularization version makes a lot of w values become zeros, which makes the model size smaller.
- L2 regularization is being used much more often.
- lambda here is the regularization parameter (hyperparameter)

FSB

L2 Regularization

Logistic regression

$w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

λ = regularization parameter

$$\min_{w,b} J(w,b)$$

$$J(w,b) = \underbrace{\frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})} + \frac{\lambda}{2m}\underbrace{\|w\|_2^2}$$

$$\cancel{+\frac{\lambda}{2m} b^2}$$

omit

L_2 regularization $\quad \|w\|_2^2 = \sum_{j=1}^{n_x} w_j^2 = w^T w \leftarrow$

L_1 regularization $\quad \frac{\lambda}{2m}\sum_{j=1}^{n_x} |w_j| = \frac{\lambda}{2m}\|w\|_1$

w will be sparse

L2 Regularization

Neural network

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \underbrace{\frac{1}{m}\sum_{i=1}^{n} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})} + \frac{\lambda}{2m}\sum_{l=1}^{L} \|w^{[l]}\|_F^2$$

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$$

$w: (n^{[l-1]}, n^{[l]})$.

"Frobenius norm"

$\|\cdot\|_2^2$ $\|\cdot\|_F^2$

L2 Regularization

Neural network

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \underbrace{\frac{1}{m}\sum_{i=1}^{n} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})} + \frac{\lambda}{2m}\sum_{l=1}^{L} \|w^{[l]}\|_F^2$$

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$$

$w: (n^{[l-1]}, n^{[l]})$

"Frobenius norm"

$\|\cdot\|_2^2$ $\|\cdot\|_F^2$

$$dw^{[l]} = \underline{(\text{from backprop}) + \frac{\lambda}{m} w^{[l]}}$$

$$\rightarrow w^{[l]} := w^{[l]} - \alpha\, dw^{[l]}$$

$$\underline{\frac{\partial J}{\partial w^{[l]}}} = dw^{[l]}$$

L2 Regularization – “weight decay”

Neural network

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \underbrace{\frac{1}{m}\sum_{i=1}^{n} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})} + \underbrace{\frac{\lambda}{2m}\sum_{l=1}^{L} \|w^{[l]}\|_F^2}$$

$$\|w^{[l]}\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$$

$w: (n^{[l-1]}, n^{[l]})$.

“Frobenius norm”

$\|\cdot\|_2^2$ $\|\cdot\|_F^2$

$$dw^{[l]} = \boxed{(\text{from backprop}) + \frac{\lambda}{m} w^{[l]}}$$

$$\rightarrow w^{[l]} := w^{[l]} - \alpha\, dw^{[l]}$$

$$\frac{\partial J}{\partial w^{[l]}} = dw^{[l]}$$

“Weight decay”

$$w^{[l]} := w^{[l]} - \alpha\left[(\text{from backprop}) + \frac{\lambda}{m} w^{[l]}\right]$$

$$= \boxed{\left(1 - \frac{\alpha\lambda}{m}\right) w^{[l]}} = \boxed{w^{[l]} - \frac{\alpha\lambda}{m} w^{[l]}} - \alpha(\text{from backprop})$$

Andrew Ng

L2 Regularization for NN

- The normal cost function that we want to minimize is:

  $J(W_1,b_1...,W_L,b_L) = (1/m) * Sum(L(y(i),y'(i)))$

- The L2 regularization version:

  $J(w,b) = (1/m) * Sum(L(y(i),y'(i))) + (lambda/2m) * Sum((||W[l]||^2)$

- We stack the matrix as one vector (mn,1) and then we apply sqrt(w1^2 + w2^2.....)

- To do back propagation (old way):

  dw[l] = (from back propagation)

- The new way:

  dw[l] = (from back propagation) + lambda/m * w[l]

FSB

L2 Regularization for NN

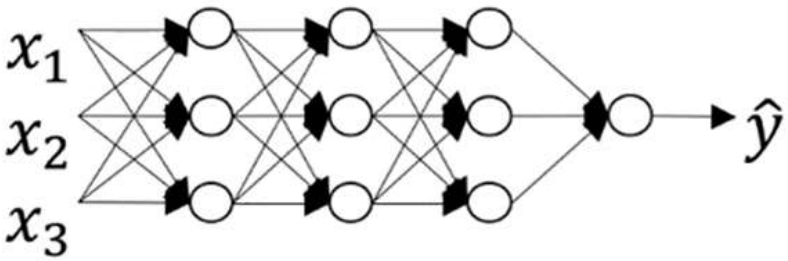
- So plugging it in weight update step:

  *w[l] = w[l] - learning_rate * dw[l]*
  *= w[l] - learning_rate * ((from back propagation) + lambda/m * w[l])*
  *= w[l] - (learning_rate*lambda/m) * w[l] - learning_rate * (from back propagation)*
  *= (1 - (learning_rate*lambda)/m) * w[l] - learning_rate * (from back propagation)*
- In practice this penalizes large weights and effectively limits the freedom in your model.

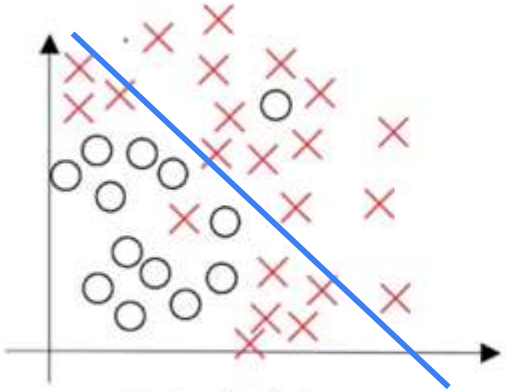
- The new *term (1 - (learning_rate*lambda)/m) * w[l]* causes the **weight to decay** in proportion to its size.

FSB

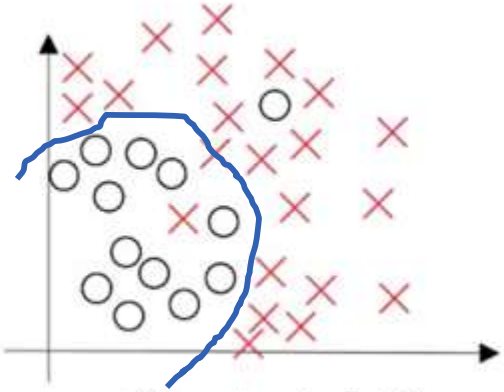
How does regularization prevent overfitting?



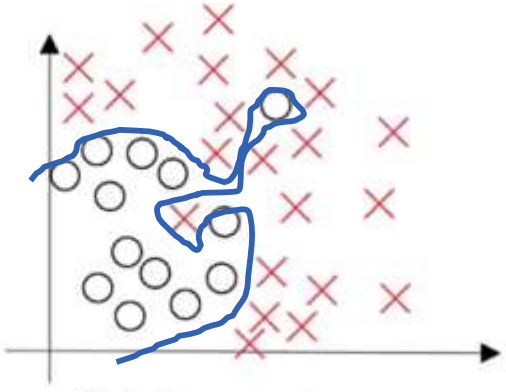
$$J(W^{[1]}, b^{[1]}) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m}\sum_{l=1}^{L} \|W^{[l]}\|_F^2$$



high bias

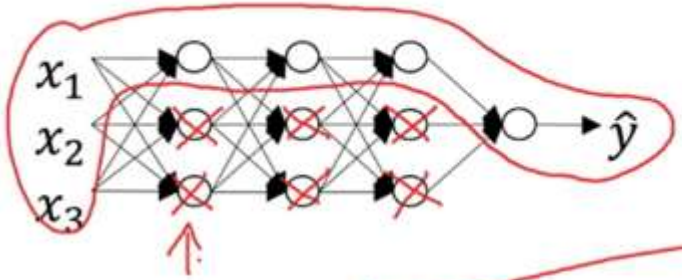


"just right"



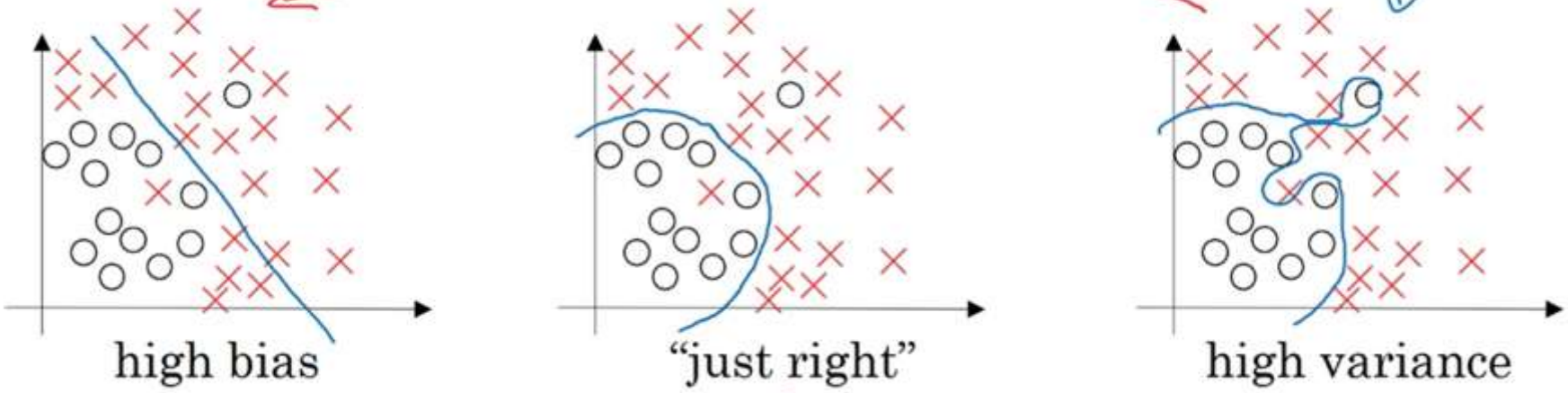
high variance

How does regularization prevent overfitting?

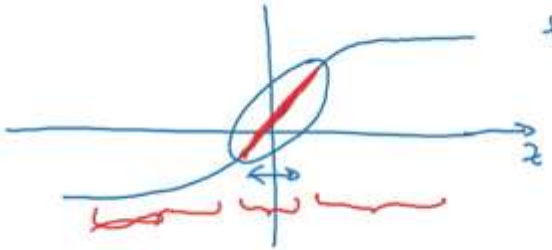


$$J(w^{[1]}, b^{[1]}) = \frac{1}{m}\sum_{i=1}^{n} \mathcal{L}(y^{(i)}, \hat{y}^{(i)}) + \frac{\lambda}{2m}\sum_{l=1}^{L} \|w^{[l]}\|_F^2$$

$$w^{[l]} \approx 0$$



How does regularization prevent overfitting?



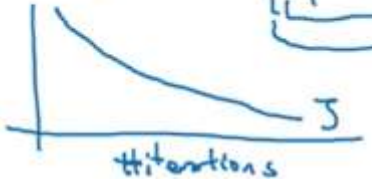
tanh

$g(z) = \tanh(z)$

$\lambda \uparrow \quad W^{[l]} \downarrow \qquad z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$

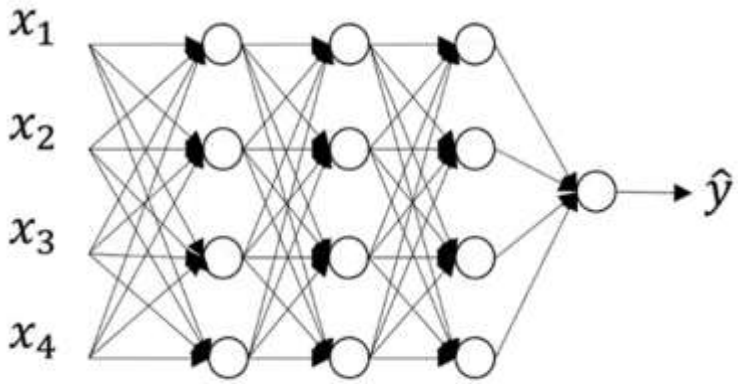
Every layer ≈ linear.

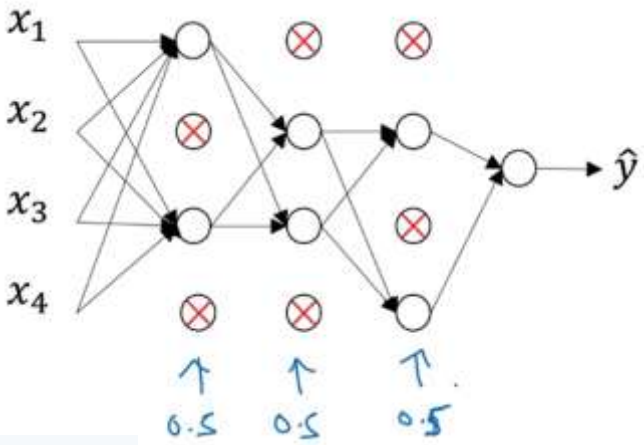
$$J(\ldots) = \boxed{\sum_i \mathcal{L}(\hat{y}^{(i)}, y^{(i)})} + \frac{\lambda}{2m} \sum_l \|W^{[l]}\|_F^2$$



FSB

Dropout regularization



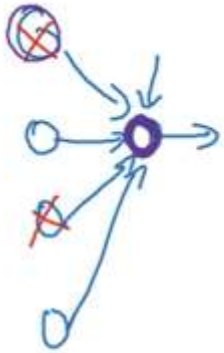


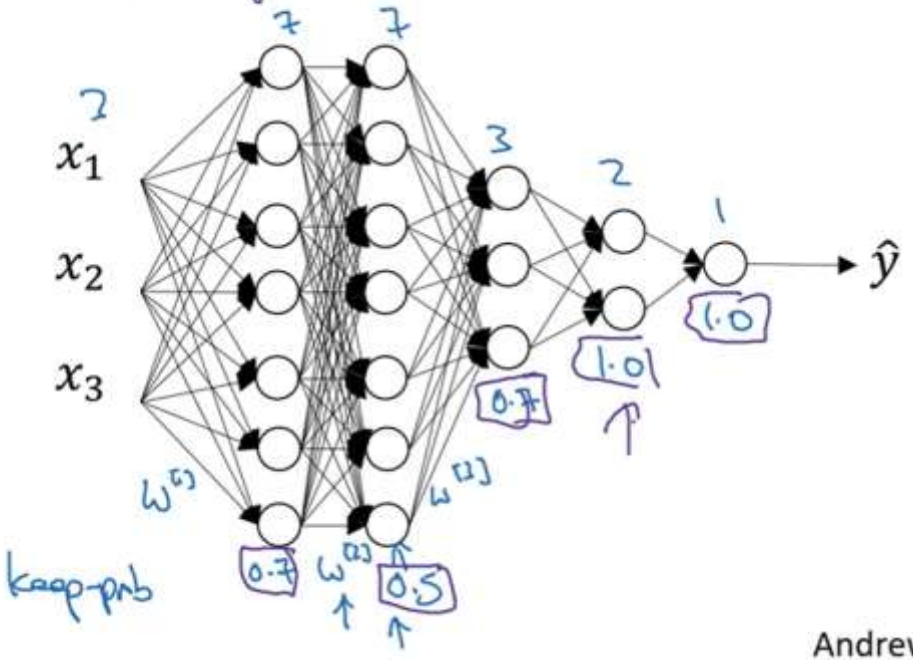

```
keep_prob = 0.8   # 0 <= keep_prob <= 1
l = 3  # this code is only for layer 3
# the generated number that are less than 0.8 will be dropped. 80% stay, 20% dropped
d3 = np.random.rand(a[l].shape[0], a[l].shape[1]) < keep_prob

a3 = np.multiply(a3,d3)   # keep only the values in d3

# increase a3 to not reduce the expected value of output
# (ensures that the expected value of a3 remains the same) - to solve the scaling problem
a3 = a3 / keep_prob
```

FSB

Why does dropout work?

Intuition: Can't rely on any one feature, so have to spread out weights. ⇝ Shrink weights. L_2





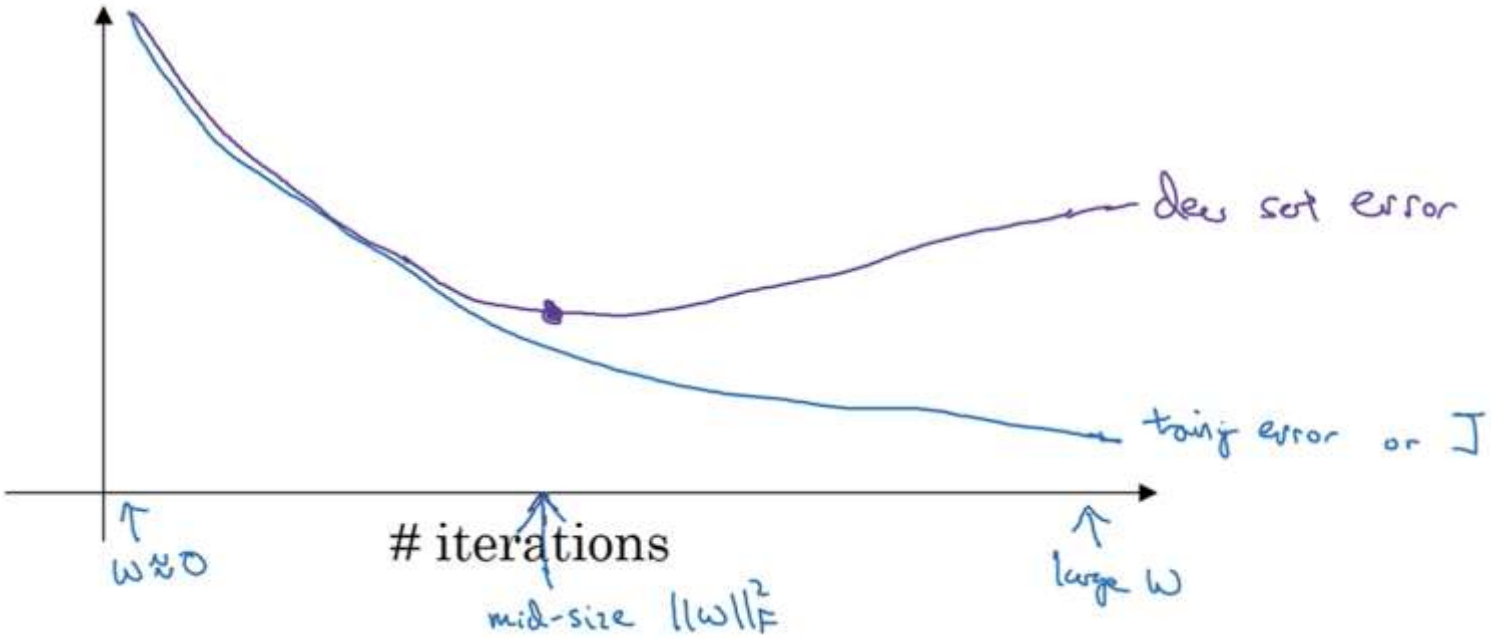
Andrew Ng

Other regularization methods

- **Data augmentation** => getting more data

Other regularization methods

- **Early stopping**



Early stopping

- In this technique we plot the training set and the dev set cost together for each iteration. At some iteration the dev set cost will stop decreasing and will start increasing.
- We will pick the point at which the training set error and dev set error are best (lowest training cost with lowest dev cost).
- We will take these parameters as the best parameters.

- Andrew prefers to use L2 regularization instead of early stopping because this technique simultaneously tries to minimize the cost function and not to overfit which contradicts the orthogonalization approach (will be discussed further).
- But its advantage is that you don't need to search a hyperparameter like in other regularization approaches (like lambda in L2 regularization).

FSB

Model Ensembles

- Algorithm:
  - Train multiple independent models.
  - At test time average their results.
- It can get you extra 2% performance.
- It reduces the generalization error.
- You can use some snapshots of your NN at the training ensembles them and take the results.

FSB

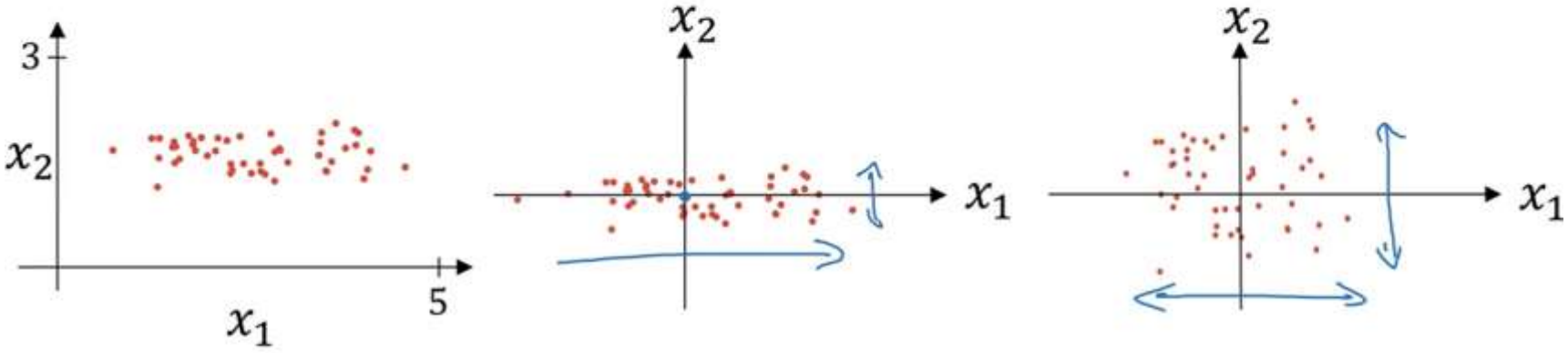
Normalizing Inputs

- If you normalize your inputs this will speed up the training process a lot.
- Normalization are going on these steps:
  1. Get the mean of the training set: *mean = (1/m) * sum(x(i))*
  2. Subtract the mean from each input: *X = X - mean*
  3. This makes your inputs centered around 0.
  4. Get the variance of the training set: variance = (1/m) * sum(x(i)^2)
  5. Normalize the variance. *X /= variance*
- These steps should be applied to training, dev, and testing sets (but using mean and variance of the train set).
- Why normalize?
  - If we don't normalize the inputs our cost function will be deep and its shape will be inconsistent (elongated) then optimizing it will take a long time.
  - But if we normalize it the opposite will occur. The shape of the cost function will be consistent (look more symmetric like circle in 2D example) and we can use a larger learning rate alpha - the optimization will be faster.

FSB

Normalizing training sets

$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$



Subtract mean:

$\boxed{\mu} = \frac{1}{m} \sum_{i=1}^{m} x^{(i)}$

$\boxed{x := x - \mu}$

Normalize variance

$\boxed{\sigma^2} = \frac{1}{m} \sum_{i=1}^{m} x^{(i)} ** 2$

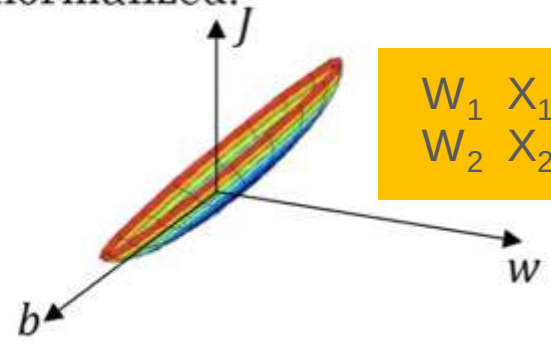
← element-wise

$\boxed{x /= \sigma^2}$

Use same μ, σ^2 to normalize test set.

Why normalize input?

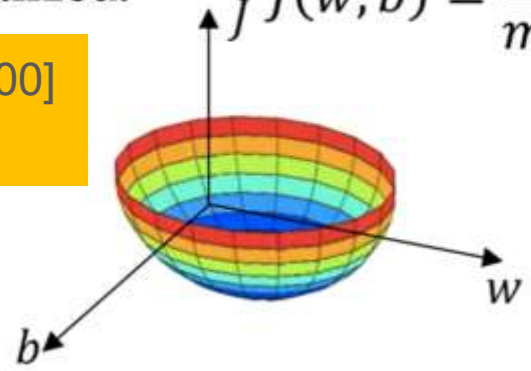
Unnormalized:

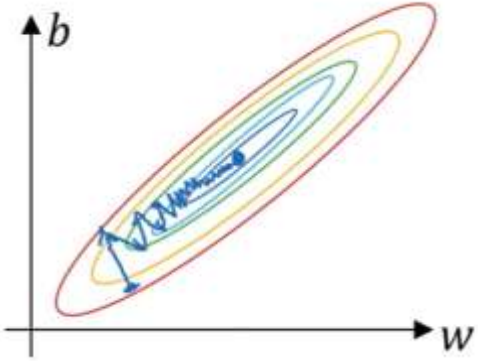


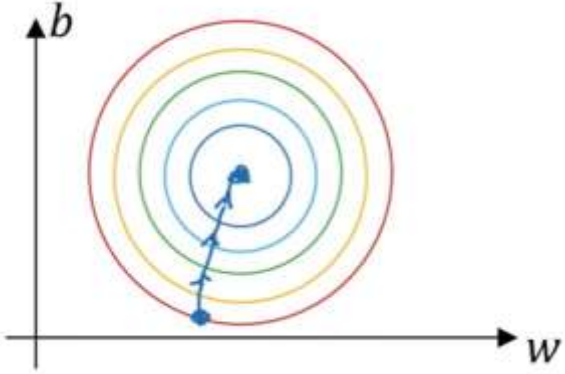
W_1 X_1: [1, …1000]
W_2 X_2: [0, …1]

Normalized:

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right)$$

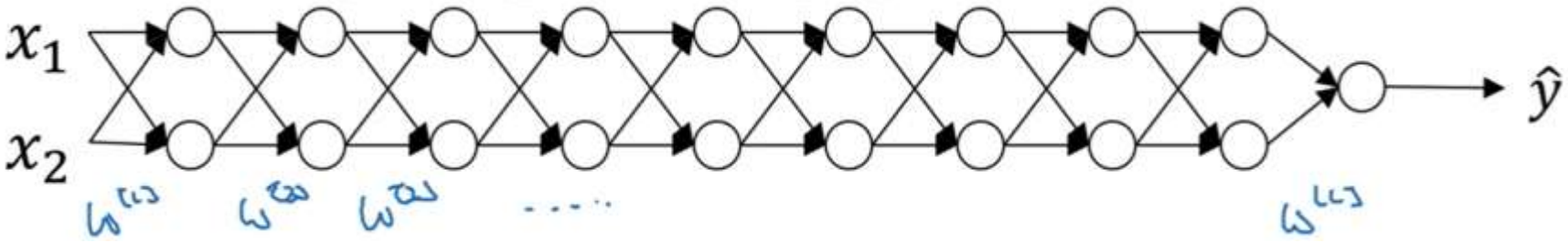






FSB

Vanishing/exploding gradients



$g(z) = z$. $\quad b^{[l]} = 0$.

$y = W^{[L]} W^{[L-1]} W^{[L-2]} \cdots$

$$\boxed{W^{[3]} \boxed{W^{[2]} W^{[1]} x}} \leftarrow a^{[1]}$$

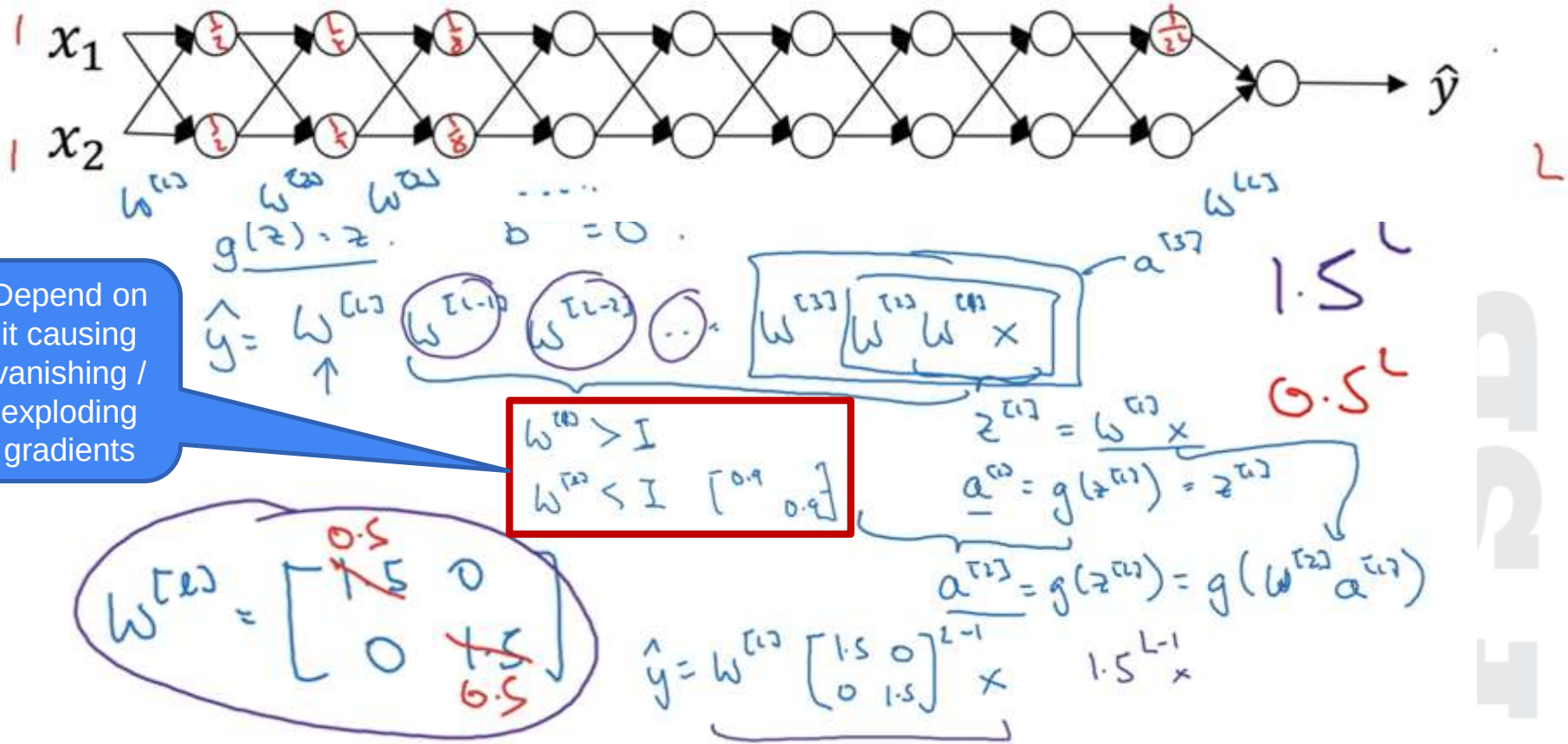
$$z^{[1]} = W^{[1]} x$$

$$a^{[1]} = g(z^{[1]}) = z^{[1]}$$

$$a^{[2]} = g(z^{[2]}) = g(W^{[2]} a^{[1]})$$

The Vanishing / Exploding gradients occurs when your derivatives become very small or very big.

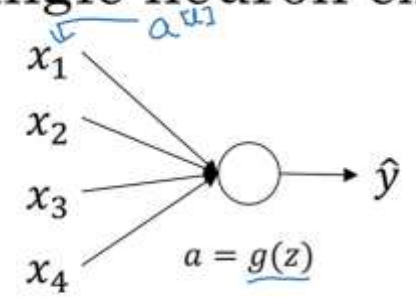
Vanishing/exploding gradients



Depend on it causing vanishing / exploding gradients

Weight initialization for DNNs

Single neuron example



$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$ $\cancel{+b}$

Large n $\rightarrow$ Smaller w_i

$Var(w_i) = \frac{1}{n}$

$W^{[l]} = np.random.randn(shape) * np.sqrt\left(\frac{1}{n^{[l-1]}}\right)$

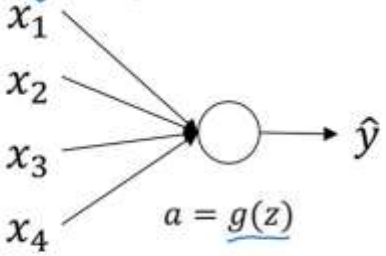
ReLU

FSB

Weight initialization for DNNs

Single neuron example

$a^{[l]}$



$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$ $\cancel{+b}$

Large n → Smaller w_i

$Var(w_i) = \frac{1}{n}$ $\quad \frac{2}{n}$

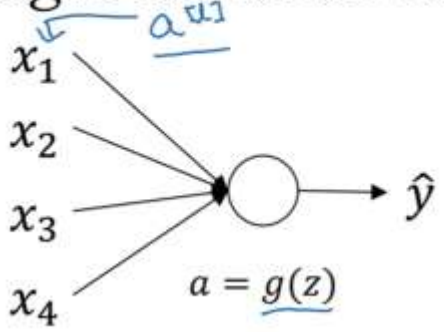
$W^{[l]} = np.random.randn(shape) * np.sqrt\left(\frac{2}{n^{[l-1]}}\right)$

ReLU $\quad g^{[l]}(z) = ReLU(z)$

FSB

Weight initialization for DNNs

Single neuron example



$w^{[l]}$

$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$ $\cancel{+b}$

large n → smaller w_i

$Var(w_i) = \frac{1}{n} \quad \frac{2}{n}$

$w^{[l]} = np.random.randn(shape) * \boxed{np.sqrt\left(\frac{2}{n^{[l-1]}}\right)}$

ReLU $\quad g^{[l]}(z) = ReLU(z)$

Other variants:

tanh $\quad \boxed{\frac{1}{n^{[l-1]}}}$

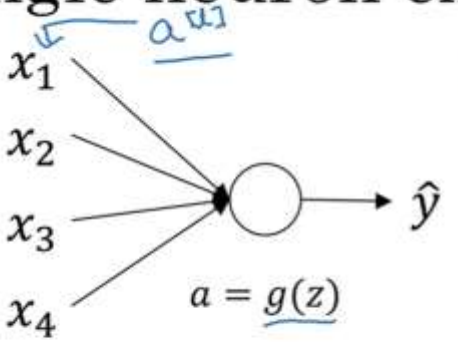
Xavier initialization

Andrew Ng

FSB

Weight initialization for DNNs

Single neuron example



$w^{[l]}$

$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$

large n → smaller w_i

$Var(w_i) = \frac{1}{n} \quad \frac{2}{n}$

$w^{[l]} = np.random.randn(shape) * \boxed{np.sqrt\left(\frac{2}{n^{[l-1]}}\right)}$

ReLU $\quad g^{[l]}(z) = ReLU(z)$

Other variants:

tanh $\quad \boxed{\sqrt{\frac{1}{n^{[l-1]}}}}$

Xavier initialization

$\sqrt{\frac{2}{n^{[l-1]} + n^{[l]}}}$

"He Initialization / Xavier Initialization"

Andrew Ng

Warm up: Initialization

- The weights $W^{[l]}$ should be initialized randomly to break symmetry
- It is however okay to initialize the biases $b^{[l]}$ to zeros. Symmetry is still broken so long as $W^{[l]}$ is initialized randomly
- Different initializations lead to different results
- Random initialization is used to break symmetry and make sure different hidden units can learn different things
- Don't intialize to values that are too large
- He initialization works well for networks with ReLU activations.

FSB

Warm up: L2 Regularization

- Observations:
  - The value of λ is a hyperparameter that you can tune using a dev set.
  - L2 regularization makes your decision boundary smoother. If λ is too large, it is also possible to "oversmooth", resulting in a model with high bias.
- What is L2-regularization actually doing?:
  - L2-regularization relies on the assumption that a model with small weights is simpler than a model with large weights. Thus, by penalizing the square values of the weights in the cost function you drive all the weights to smaller values. It becomes too costly for the cost to have large weights! This leads to a smoother model in which the output changes more slowly as the input changes.

FSB

Warm up: L2 Regularization

What you should remember: Implications of L2-regularization on:

- cost computation:
  - A regularization term is added to the cost
- backpropagation function:
  - There are extra terms in the gradients with respect to weight matrices
- weights:
  - weights end up smaller ("weight decay") - are pushed to smaller values.

FSB

Warm up: Dropout

- What you should remember about dropout:

- Dropout is a regularization technique.
- You only use dropout during training. Don't use dropout (randomly eliminate nodes) during test time.
- Apply dropout both during forward and backward propagation.
- During training time, divide each dropout layer by keep_prob to keep the same expected value for the activations. For example, if keep_prob is 0.5, then we will on average shut down half the nodes, so the output will be scaled by 0.5 since only the remaining half are contributing to the solution. Dividing by 0.5 is equivalent to multiplying by 2. Hence, the output now has the same expected value. You can check that this works even when keep_prob is other values than 0.5.

Practice with Regularization

https://github.com/ashishpatel26/Andrew-NG-Notes/blob/master/Deep%20Learning%20Notebooks%20by%20Andrew%20NG/Improving%20Deep%20Neural%20Networks%20Hyperparameter%20tuning%2C%20Regularization%20and%20Optimization/Regularization.ipynb

FSB

CHALLENGE OF CLASSIFICATION

Option 1

https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

Option 2

https://www.kaggle.com/code/vencerlanz09/98-fire-classification-cnn-mobilenetv2/notebook

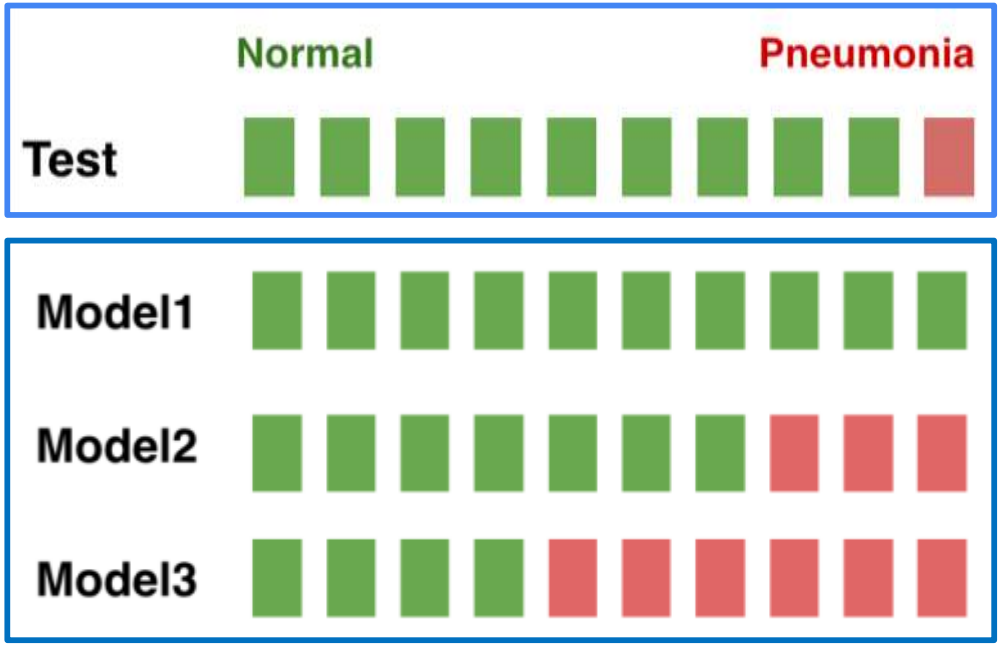
FSB

Evaluation metrics

FSB

Classification evaluation metrics

- Let's consider a simple binary classification problem, where we are trying to predict if a patient is healthy or has pneumonia. We have a test set with 10 patients, where 9 patients are healthy (shown as green squares) and 1 patient has pneumonia (shown as a red square).



FSB

Accuracy

To compare the models, we could first use accuracy, which is the number of correctly classified examples divided by the total:

$$\text{accuracy}(f) = \frac{\sum_{x_i \in X_{test}} 1\{f(x_i) = y_i\}}{|X_{test}|}$$

If we use accuracy as your evaluation metric, it seems that the best model is Model1.

$$\text{Accuracy}(M_1) = \frac{9}{10} \qquad \text{Accuracy}(M_2) = \frac{8}{10} \qquad \text{Accuracy}(M_3) = \frac{5}{10}$$

In general, when you have class imbalance (which is most of the time!), accuracy is not a good metric to use.

FSB

Confusion matrix

- Accuracy doesn't discriminate between errors (i.e., it treats misclassifying a patient with pneumonia as healthy the same as misclassifying a visualizing patient with having pneumonia). A confusion matrix is a tabular format for showing a more detailed breakdown of a model's correct and incorrect classifications.

		Actual	
		+	-
Predicted	+	True Positive	False Positive
	-	False Negative	True Negative

FSB

Precision, Recall, and the F1 Score

Out of all the patients that are predicted to have pneumonia, how many actually have pneumonia?

This metric is precision, expressed as the following:

$$\text{Precision} = \frac{tp}{tp + fp}$$

What is the precision for each model?

$$\text{precision}(M_1) = \frac{0}{0} \qquad \text{precision}(M_2) = \frac{1}{3} \qquad \text{precision}(M_3) = \frac{1}{6}$$

FSB

Precision, Recall, and the F1 Score

Out of all the patients with pneumonia, how many did the model predict as having pneumonia?

This metric is recall, which is expressed as the following:

$$\text{Recall} = \frac{tp}{tp + fn}$$

What is the recall for each model?

$$\text{Recall}(M_1) = \frac{0}{1} \qquad \text{Recall}(M_2) = \frac{1}{1} \qquad \text{Recall}(M_3) = \frac{1}{1}$$

FSB

Precision, Recall, and the F1 Score

"Having multiple-number evaluation metrics makes it harder to compare algorithms. Better to combine them to a single evaluation metric. Having a single-number evaluation metric speeds up your ability to make a decision when you are selecting among a large number of classifiers. It gives a clear preference ranking among all of them, and therefore a clear direction for progress." - Machine Learning Yearning

F1 score is a metric that combines recall and precision by taking their harmonic mean:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

What is the F1 score for each model?

$$F_1(M_1) = 0 \qquad F_1(M_2) = \frac{1}{2} \qquad F_1 = \frac{2}{7}$$

If F1 score is a great one-number measurement of model performance, why don't we use it as the loss function?